

Dynamic Provider Discovery Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking. Every test commit carries the Anti-Bluff Bluff-Audit: stamp (Mutation/Observed/Reverted) per §6.J/§6.L. Commit + push after each task; keep scripts/check-constitution.sh GREEN.

Goal: The Android client populates its provider list AND per-provider auth/sign-in UI dynamically from the chosen API instance; every configured Jackett indexer becomes a first-class discoverable provider usable with all client features.

Architecture: Add GET /v1/providers to lava-api-go returning the full provider catalogue (native + one entry per Jackett indexer). The client fetches it, maps each entry to a RemoteTrackerDescriptor, and registers a generic ApiBackedTrackerClient per provider that routes every operation through /v1/{id}/{op}. The compiled-in 7-provider registry becomes an offline fallback.

Tech Stack: Go 1.26 (Gin, oapi-codegen), Kotlin/Jetpack Compose, Orbit MVI, Hilt, kotlinx-serialization, Retrofit/OkHttp network layer, JUnit4 + MockWebServer, Robolectric, Compose UI Challenge tests on Genymotion via Containers submodule, HelixQA.

Spec: docs/superpowers/specs/2026-06-11-dynamic-provider-discovery-design.md

Phase 0 — Submodule sync (deliberate, build-verified)

Task 0.1: Pull leaf code-dependency submodules

Files: submodules/
{auth,cache,concurrency,config,database,ratelimiter,recovery,security,http3,mdns,middleware,observability} (gitlinks), .gitmodules (unchanged)



Step 1: For each leaf submodule: `git -C submodules/<x> fetch origin && git -C submodules/<x> log --oneline HEAD..origin/HEAD | head — record what's incoming.`



Step 2: `git -C submodules/<x> pull --ff-only` for each. If a non-ff is needed, STOP and report (don't force).



Step 3: Rebuild the Go module: `cd lava-api-go && go build ./... && GOMAXPROCS=2 nice -n 19 go test ./internal/... 2>&1 | tail -20`. Expected: build OK, tests pass.



Step 4: If green, stage the pin bumps: `git add submodules/<x> ...; commit chore(submodules): pull leaf deps to latest (build+test green).`



Step 5: Push to github + gitlab.

Task 0.2: Pull containers + challenges + helixqa + tracker_sdk



Step 1: `git -C submodules/containers pull --ff-only; git -C submodules/challenges pull --ff-only; git -C submodules/helixqa pull --ff-only (per the always-track-upstream waiver, §6.AD); git -C submodules/tracker_sdk pull --ff-only.`



Step 2: Rebuild Android: `./gradlew :app:assembleDebug -x lint 2>&1 | tail -5`. Rebuild Go. Run `scripts/run-genymotion-challenges.sh --test-class lava.app.challenges.Challenge00CrashSurvivalTest` (if a VM is up) as a smoke gate.



Step 3: On green, commit the pin bumps + push.



Step 4: constitution + panoptic: do NOT pull here. Note in CONTINUATION that they advance via CONST-049 separately; flag any new mandatory rules for a follow-up adoption cycle.

Phase 1 — API: provider catalogue endpoint

Task 1.1: Extend the Provider interface with catalogue metadata

Files:

- Modify: lava-api-go/internal/provider/provider.go (the Provider interface ~244-275)
- Test: lava-api-go/internal/provider/provider_meta_test.go



Step 1 (test first): Write `TestProvider_CatalogueMetadataDefaults` asserting a minimal fake provider exposes `Kind() == "native"`, `SupportsAnonymous() == false`, `BaseURLs() == nil` by default.



Step 2: Run `cd lava-api-go && go test ./internal/provider/ -run TestProvider_CatalogueMetadata -v`. Expected: FAIL (methods undefined).



Step 3: Add to the Provider interface: `Kind() string`, `SupportsAnonymous() bool`, `BaseURLs() []string`. Provide a `BaseProvider` embeddable struct (or default helper) so existing providers compile with defaults (`Kind()="native"`). Update existing native provider impls (rutracker, rutor, nnmclub, kinozal, archiveorg, gutenbergl, iptorrents) to return their real BaseURLs + `SupportsAnonymous`.



Step 4: `go build ./... && go test ./internal/provider/ -v`. Expected: PASS.



Step 5: Commit with Bluff-Audit stamp; push.

Task 1.2: GET /v1/providers handler

Files:

- Create: lava-api-go/internal/handlers/v1/providers.go
- Test: lava-api-go/internal/handlers/v1/providers_test.go
- Modify: lava-api-go/internal/handlers/v1/handlers.go (register route before the `/:provider/` group)
- Modify: lava-api-go/api/openapi.yaml (add the path + `ProviderDescriptor` schema); regenerate `internal/gen/server`



Step 1 (test): TestProvidersHandler_ReturnsCatalogue — real http test Gin engine with a registry holding 2 fake providers (one native, one with kind=jackett); assert GET /v1/providers 200 + JSON body has both, with fields id,displayName,kind,capabilities,authType,encoding,baseUrls,supportsAnonymous (and indexer for the jackett one). Primary assertion on the response BODY (§6.AB), not the status alone.



Step 2: Run it → FAIL (handler undefined).



Step 3: Implement ProvidersHandler.GetProviders(c *gin.Context) building the DTO list from registry.All(); map capability enums → string names; §6.AC telemetry on any error. Register at group.GET("/providers", providers.GetProviders) BEFORE the /:provider middleware group in handlers.go. Add the path to openapi.yaml; make generate (oapi-codegen) → ensure empty diff gate still passes.



Step 4: go test ./internal/handlers/v1/ -run TestProviders -v → PASS. go test ./internal/... → PASS.



Step 5: Commit (Bluff-Audit) + push.

Task 1.3: Contract test — discovery over real HTTP

Files: Create lava-api-go/tests/contract/providers_contract_test.go



Step 1 (test): Boot the real router (real registry, no Postgres needed for discovery), GET /v1/providers, assert every returned provider's declared capabilities each resolve to a non-501 on /v1/{id}/{cap-route} (capability honesty, §6.E) — at least the SEARCH route returns != 501/404 for each.



Step 2: go test ./tests/contract/ -run TestProvidersContract -v → FAIL.



Step 3: (No new prod code — this validates Task 1.2.) If it fails because a provider declares a cap with no route, fix the registration/declaration.



Step 4: → PASS. Commit + push.

Phase 2 — API: Jackett indexers as providers

Task 2.1: Enumerate Jackett indexers

Files:

- Create: `lava-api-go/internal/jackett/indexers.go`
- Test: `lava-api-go/internal/jackett/indexers_test.go` (+ testdata fixture of Jackett `/api/v2.0/indexers` JSON)



Step 1 (test): `TestListIndexers_ParsesConfigured` — MockWebServer-style httptest server returning a captured Jackett indexers JSON; assert `ListIndexers(ctx)` returns the expected `[]IndexerInfo{ID,Name,Caps}` (configured indexers only). Real HTTP (not mocked client).



Step 2: Run → FAIL.



Step 3: Implement `Client.ListIndexers(ctx) ([]IndexerInfo, error) → GET {base}/api/v2.0/indexers?configured=true&apikey=...` (apikey from config, §6.H), parse JSON.



Step 4: → PASS. Commit (Bluff-Audit) + push.

Task 2.2: JackettIndexerProvider implements Provider

Files:

- Create: `lava-api-go/internal/provider/jackettprovider/provider.go`
- Test: `lava-api-go/internal/provider/jackettprovider/provider_test.go`



Step 1 (test): `TestJackettIndexerProvider_SearchDelegates` — a `JackettIndexerProvider{indexer:"1337x", client:<fake jackett searcher>}`; assert `ID()=="1337x"`, `Kind()=="jackett"`, `AuthType()==AuthNone`, `Capabilities()` contains `SEARCH+MAGNET_LINK+TORRENT_DOWNLOAD`; `Search(...)` calls the jackett client with indexer "1337x" and maps to `provider.SearchResult`. `GetForumTree` etc. return the not-implemented error.



Step 2: Run → FAIL.



Step 3: Implement the type. Search/DownloadFile delegate to the jackett client; extended caps return provider.ErrNotImplemented.



Step 4: → PASS. Commit + push.

Task 2.3: Register a provider per indexer at startup (with collision guard)

Files:

- Modify: lava-api-go/internal/router/router.go (the JackettEnabled wiring block ~117-125)
- Test: lava-api-go/internal/router/jackett_registration_test.go



Step 1 (test): TestJackettProvidersRegistered — build the router with Jackett enabled + a stub ListIndexers returning ["1337x", "rutracker"]; assert the registry now contains 1337x as a jackett provider AND that rutracker (collision with a native id) was NOT overwritten (native wins) + a warning recorded.



Step 2: Run → FAIL.



Step 3: In router wiring: when Jackett enabled, ListIndexers → for each, if registry.Get(id)==nil register a JackettIndexerProvider; else skip + observability.RecordWarning. Enumeration failure → log \$6.AC, continue with native providers.



Step 4: → PASS; GET /v1/providers now includes jackett indexers (extend Task 1.2 test data via an integration check). Commit + push.

Task 2.4: e2e — discover → search a Jackett provider

Files: Create lava-api-go/tests/e2e/jackett_provider_e2e_test.go



Step 1 (test): Real router + a stub Jackett upstream (httptest serving Torznab XML); GET /v1/providers lists the indexer; GET /v1/{indexer}/search?query=ubuntu returns mapped results; GET /v1/{indexer}/download/{id} returns the magnet/torrent. Assert on the result body.



Step 2-4: Run → fix wiring if needed → PASS. Commit + push.

Phase 3 — Client: catalogue fetch + remote descriptor

Task 3.1: ProviderDescriptorDto + JSON parse

Files:

- Create: `core/network/src/main/kotlin/lava/network/dto/ProviderDescriptorDto.kt` (or the existing dto package)
- Test: `core/network/src/test/kotlin/lava/network/dto/ProviderDescriptorDtoTest.kt`



Step 1 (test): Parse a captured GET `/v1/providers` JSON (native + jackson entries) → assert fields incl. kind, optional indexer, capabilities list, authType, supportsAnonymous.



Step 2: Run `./gradlew :core:network:test --tests "*ProviderDescriptorDtoTest"` → FAIL.



Step 3: @Serializable data class `ProviderDescriptorDto(...)` + data class `ProvidersResponseDto(val providers: List<ProviderDescriptorDto>)`.



Step 4: → PASS. Commit (Bluff-Audit) + push.

Task 3.2: RemoteTrackerDescriptor mapping

Files:

- Create: `core/tracker/api/src/main/kotlin/lava/tracker/api/RemoteTrackerDescriptor.kt`
- Test: `core/tracker/api/src/test/kotlin/lava/tracker/api/RemoteTrackerDescriptorTest.kt`



Step 1 (test): Given a `ProviderDescriptorDto`, `RemoteTrackerDescriptor.from(dto)` yields a `TrackerDescriptor` with `trackerId`, `displayName`, `capabilities: Set<TrackerCapability>` (string→enum, unknown values dropped with a logged warning, NOT a crash), `authType: AuthType`, `baseUrls: List<MirrorUrl>`, `apiSupported=true`, `verified=true`, `supportsAnonymous` from dto. Assert an unknown capability string does not throw.



Step 2-4: FAIL → implement mapping (tolerant enum parse) → PASS. Commit + push.

Task 3.3: ProviderCatalogRepository + FetchProvidersUseCase

Files:

- Create: core/data/src/main/kotlin/lava/data/provider/ProviderCatalogRepository.kt
- Create: core/domain/src/main/kotlin/lava/domain/usecase/FetchProvidersUseCase.kt
- Test: core/data/src/test/kotlin/lava/data/provider/ProviderCatalogRepositoryTest.kt (MockWebServer)



Step 1 (test): MockWebServer serving /v1/providers; repository.fetchProviders(baseUrl) returns the mapped List<RemoteTrackerDescriptor>. Add a failure case: 500 / timeout → returns Result.failure (or empty + flag), does NOT throw. Real OkHttp, real parse (§6.J — not mocked SUT).



Step 2-4: FAIL → implement repo (real network call + parse + map) + the use case wrapping it → PASS. Commit + push.

Phase 4 — Client: generic API-backed client + dynamic registry

Task 4.1: ApiBackedTrackerClient

Files:

- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/ApiBackedTrackerClient.kt
- Test: core/tracker/client/src/test/kotlin/lava/tracker/client/ApiBackedTrackerClientTest.kt (MockWebServer)



Step 1 (test): Construct with a RemoteTrackerDescriptor (caps={SEARCH,TORRENT_DOWNLOAD}, authType=NONE) + a network api pointed at MockWebServer. Assert: getFeature(SearchableTracker::class) non-null; search(q) issues GET /v1/{id}/search?query=q and maps the response to domain SearchResult; getFeature(BrowsableTracker::class) is NULL (cap not declared — capability honesty); download(id) hits /v1/{id}/download/{id}. Primary assertion: the request path + the parsed result (§6.AB).



Step 2-4: FAIL → implement the client delegating each feature to the network layer keyed by `descriptor.trackerId`; `getFeature<T>` gated on `descriptor.capabilities` → PASS. Commit + push.

Task 4.2: `DynamicTrackerRegistry.populateFrom`

Files:

- Modify: `core/tracker/registry/.../DefaultTrackerRegistry.kt` (add `populateFrom(descriptors, clientFactory)`)
- Modify: `core/tracker/client/.../di/TrackerClientModule.kt` (registry becomes runtime-populatable; bundled 7 are the fallback)
- Test: `core/tracker/registry/src/test/kotlin/.../DynamicRegistryTest.kt`



Step 1 (test): `populateFrom([descA, descB])` registers an `ApiBackedTrackerClient` for each; `listAvailableTrackers()` returns `A+B`; re-populate replaces; empty list → falls back to bundled descriptors (assert the 7 bundled appear when populate is given empty).



Step 2-4: FAIL → implement → PASS. Commit + push.

Phase 5 — Client: onboarding wiring + auth UI

Task 5.1: Onboarding fetches the catalogue after API selection

Files:

- Modify: `feature/onboarding/.../OnboardingViewModel.kt` (after `ApiSelection` probe success → `FetchProvidersUseCase` → populate registry → advance)
- Test: `feature/onboarding/src/test/kotlin/.../OnboardingViewModelDynamicProvidersTest.kt`



Step 1 (test): Real `OnboardingViewModel` + real `FetchProvidersUseCase` + `MockWebServer` API; drive `ApiSelection`→probe→fetch; assert `state.providers` reflects the API's list (incl. a provider the bundled registry does NOT have, e.g. a `jackett` indexer); assert fetch-failure path shows the bundled fallback + a non-blocking notice (NOT a blank list — §6.AB).



Step 2-4: FAIL → wire it → PASS. Commit + push.

Task 5.2: Provider list + auth UI render from dynamic descriptors

Files:

- Modify (verify/extend): feature/login/.../
ProviderLoginViewModel.kt, ProviderLoginScreen.kt (ensure NONE/API_KEY/FORM_LOGIN/CAPTCHA all render; add API_KEY field if missing)
- Test: feature/login/src/test/kotlin/.../
ProviderLoginAuthUiTest.kt



Step 1 (test): For each AuthType, the form composable shows the right fields (NONE→Continue; FORM_LOGIN→user+pass; CAPTCHA_LOGIN→user+pass+captcha; API_KEY→key field). Robolectric/Compose test asserting node presence (rendered UI state, §6.AB clause 1).



Step 2-4: FAIL → add API_KEY rendering branch → PASS. Commit + push.

Phase 6 — Challenge tests + HelixQA

Task 6.1: Challenge39DynamicProviderDiscoveryTest

Files: Create app/src/androidTest/kotlin/lava/app/challenges/Challenge39DynamicProviderDiscoveryTest.kt



Step 1: Drive onboarding → choose the API → assert the provider list is populated FROM the API (a provider the bundled set lacks appears) → select it → search → assert a real result row renders. KDoc carries the FALSIFIABILITY REHEARSAL block (break the fetch → list falls back/empty → test fails).



Step 2: Run on the Genymotion VM: scripts/run-genymotion-challenges.sh --test-class
lava.app.challenges.Challenge39DynamicProviderDiscoveryTest.
Expected: BUILD SUCCESSFUL + PASS. Capture evidence.



Step 3: Perform the falsifiability rehearsal; record in the commit Bluff-Audit. Commit + push.

Task 6.2: Challenge40JackettIndexerProviderTest

Files: Create app/src/androidTest/kotlin/lava/app/challenges/Challenge40JackettIndexerProviderTest.kt



Step 1: A Jackett indexer appears as a provider → select → search → result → download reachable. Falsifiability block.



Step 2: Run on VM, capture evidence. Commit + push.

Task 6.3: HelixQA QA session over the dynamic onboarding

Files: Add/extend a HelixQA challenge script invoked by scripts/run-helixqa-challenges.sh



Step 1: Author a QA-session scenario: launch → onboarding → choose API → provider list from API → pick a provider → search. Vision-guided.



Step 2: bash scripts/run-helixqa-challenges.sh --only dynamic-provider-discovery --runner host (or containerized). Capture evidence under .lava-ci-evidence/.../helixqa-challenges/.



Step 3: Commit + push.

Task 6.4: coverage scanner + challenge-coverage gate



Step 1: Run scripts/check-challenge-coverage.sh — confirm the onboarding/provider features map to the new Challenges. Run scripts/check-challenge-discrimination.sh STRICT on C39/C40.



Step 2: Run scripts/check-constitution.sh GREEN. Commit any gate-doc updates.

Phase 7 — Full build, device verification, evidence, CONTINUATION

Task 7.1: Full rebuild + full test suite



Step 1: `cd lava-api-go && GOMAXPROCS=2 nice -n 19 go test ./... 2>&1 | tail -30` → all PASS.



Step 2: `./gradlew testDebugUnitTest 2>&1 | tail -20` → all PASS.



Step 3: `build_and_release.sh` → 4 APKs + api-go binary/image build. Capture.

Task 7.2: Device gate (Genymotion via Containers, §6.AH)



Step 1: `scripts/run-genymotion-challenges.sh` core set (C00/C01) + C39/C40 + the existing provider Challenges (C02/C03/C09-12/C38). All PASS, evidence captured.



Step 2: Record the per-AVD evidence.

Task 7.3: CONTINUATION + ledger + push



Step 1: Update `docs/CONTINUATION.md` §0 (feature landed, evidence paths). Record LVA items for the feature in `docs/workable_items.db`; regen Issues/Fixed; sync gate GREEN.



Step 2: Commit + push to both mirrors; verify convergence + check-constitution GREEN.



Step 3: Report to operator. (Distribute is a SEPARATE operator-gated step — do not distribute without explicit go-ahead per §6.AA/§6.Z; the auth rotation + two-stage + canary runbook applies.)

Self-review notes

- **Spec coverage:** §4.1 → Phase 1+2; §4.2 → Phase 3+4+5; §4.3 dataflow → exercised by Phase 6; §5 error handling → Tasks 3.3/5.1 (fallback) + 1.2/2.3 (telemetry); §6 testing → Phases 1-2

(Go), 3-5 (Kotlin), 6 (Challenge+HelixQA); §7 submodules → Phase 0; §9 distribute → Task 7.3 note (gated).

- **No placeholders:** every task names exact files, signatures, test intent, and run commands. Implementation bodies are produced via TDD by the executing subagent against the stated tests + the spec's concrete interfaces.
- **Type consistency:** `RemoteTrackerDescriptor.from(dto)`, `ProviderCatalogRepository.fetchProviders(baseUrl)`, `FetchProvidersUseCase`, `ApiBackedTrackerClient`, `DefaultTrackerRegistry.populateFrom`, `ProvidersHandler.GetProviders`, `JackettIndexerProvider`, `Client.ListIndexers` — names used consistently across tasks.